

TRAFFIC SIGNAL OPTIMIZER

Product Requirements Document (PRD)

Deep Learning + Computer Vision + Intelligent Traffic Signal Optimization

Project	Traffic Signal Optimizer
Project Type	Deep Learning / Computer Vision / Intelligent Optimization
Primary Language	Python
Core AI	YOLO-based vehicle detection
Decision Engine	Custom mathematical optimization algorithm
Signal Strategy	Fixed round-robin order with dynamic timing
Input	Camera snapshots
Primary Traffic Inputs	Vehicle count / CEU + direction-level waiting time

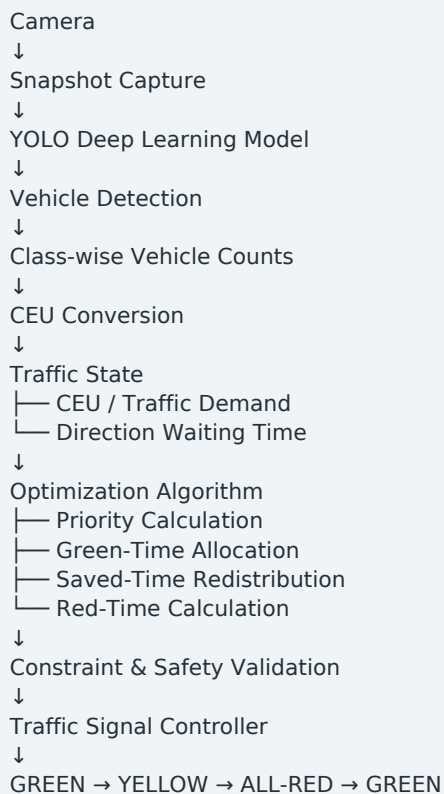
Architecture baseline: YOLO sees → traffic engine quantifies → optimizer decides → safety validates → controller executes → dashboard observes.

1. Product Overview

Traffic Signal Optimizer is an intelligent traffic-signal management system that improves a conventional round-robin traffic-control system without changing the order in which directions are served. The system uses elevated cameras to capture snapshots of traffic approaches, YOLO to detect and classify vehicles, a Car Equivalent Unit (CEU) engine to convert vehicle classes into standardized traffic demand, and a waiting-time engine to represent directional fairness. A custom optimization engine then determines future green durations and derives the corresponding upcoming red durations. A deterministic safety layer validates every timing plan before the signal controller executes it.

The system is designed around a rolling look-ahead mechanism. The active phase is not continuously recalculated. When the current green phase reaches approximately its final five seconds, the system captures fresh snapshots of all upcoming directions, processes them through YOLO, constructs the next traffic state, and calculates the future timing plan. Approved timings are frozen for execution. The current round-robin order is never skipped, reordered, or interrupted.

1.1 Core Product Flow



1.2 Core Architectural Principle

YOLO is the learned perception component. CEU conversion, waiting-time calculation, priority calculation, green allocation, redistribution, red calculation, safety validation, and signal control are implemented as explicit software logic. The optimizer is intentionally not forced into a neural-network model. This makes the timing decisions explainable and allows hard constraints and safety behavior to be guaranteed.

2. Product Goals

- Use computer vision and deep learning to detect vehicles from traffic-camera snapshots.
- Convert detected vehicle classes into a standardized CEU traffic-demand value.

- Use directional waiting time alongside CEU so that traffic demand and fairness both influence timing.
- Dynamically allocate green duration while preserving the fixed round-robin phase order.
- Dynamically determine upcoming red duration from the optimized future phase schedule.
- Use a five-second look-ahead trigger rather than continuously recalculating the active signal.
- Guarantee configurable timing constraints and a strict two-second optimization timeout.
- Provide a monitoring dashboard showing the state of the intersection and optimization system.
- Keep the architecture independent of physical signal hardware so a hardware prototype can be connected later.

3. Scope

3.1 Included

- Configurable three-way and four-way intersection support.
- Fixed round-robin order for configured approaches.
- Elevated/high-mounted camera input.
- Snapshot-based traffic observation.
- YOLO vehicle detection and class-wise counting.
- Vehicle classes: car, bike, bus, and truck.
- CEU conversion using configurable vehicle weights.
- Direction-level waiting-time tracking; no individual vehicle tracking.
- Priority calculation using CEU and waiting time.
- Green-time allocation between configured minimum and maximum limits.
- Reference green duration used as the baseline for timing decisions.
- Saved-time redistribution toward higher-priority directions when useful.
- Dynamic upcoming red-duration calculation.
- GREEN → YELLOW → ALL-RED → next GREEN state transitions.
- Deterministic timing validation and safety fallback.
- Two-second hard timeout for the optimization pipeline.
- FastAPI backend and monitoring dashboard.
- Automated software testing and optional physical hardware testing.

3.2 Explicitly Out of Core Scope

- Continuous video processing.
- Individual vehicle tracking.
- Queue-length estimation.
- Historical traffic prediction.
- Weather/event prediction.

- Reinforcement learning.
- Dynamic phase ordering.
- Automatic phase skipping.
- Mid-phase recalculation or interruption.
- Complex client-facing analytics.
- SUMO.
- OpenCV.

3.3 Future Enhancements (Optional)

- Queue-length estimation to supplement simple vehicle counts.
- Historical traffic patterns and predictive traffic modelling.
- Reinforcement-learning or other learned optimization approaches after sufficient data is available.
- Multi-intersection coordination.
- Additional sensor inputs.
- Advanced traffic analytics such as stops, travel time, fuel, or emissions estimation.
- Additional intersection topologies.
- Physical traffic-signal deployment.

4. Functional Requirements

ID	Requirement	Priority
FR-001	System shall support configurable intersection approaches and round-robin order.	Must
FR-002	System shall capture snapshots of upcoming/other approaches when the active green phase approaches approximately five seconds.	Must
FR-003	System shall pass captured images to YOLO for vehicle detection.	Must
FR-004	System shall produce class-wise counts for cars, bikes, buses, and trucks.	Must
FR-005	System shall convert class-wise counts into CEU using configurable weights.	Must
FR-006	System shall maintain direction-level waiting time.	Must
FR-007	System shall construct a standardized traffic state from CEU and waiting time.	Must
FR-008	System shall calculate a priority for each upcoming direction.	Must
FR-009	System shall allocate future green durations within configured minimum and maximum limits.	Must
FR-010	System shall redistribute useful saved time subject to maximum limits.	Must
FR-011	System shall calculate upcoming red durations from the future timing schedule and transition intervals.	Must
FR-012	System shall freeze approved future phase durations until their execution.	Must
FR-013	System shall preserve phase order and shall not skip or interrupt phases.	Must
FR-014	System shall validate every timing plan before execution.	Must
FR-015	System shall abort optimization and use fallback timing if no valid result is available within 2.0 seconds.	Must
FR-016	System shall expose operational state to the dashboard.	Should
FR-017	System shall support a hardware-independent signal interface.	Should

5. Timing Model

Parameter	Prototype Value	Purpose
Minimum Green	10 sec	Lower bound for every served direction
Reference Green	30 sec	Normal/reference baseline
Maximum Green	60 sec	Upper bound for dynamic allocation
Yellow	2.0 sec	Transition after green
All-Red	0.5 sec	Transition clearance interval
Optimization Trigger	5 sec remaining	Look-ahead calculation trigger
Optimization Timeout	2.0 sec	Hard deadline for a valid optimization result

These are configurable prototype values and are not presented as universal real-world traffic engineering or safety standards. Any real deployment would require appropriate traffic engineering, hardware, regulatory, and jurisdiction-specific validation.

6. Computer Vision and Deep Learning

6.1 Camera and Snapshot Strategy

Cameras are positioned at an elevated/high-mounted location to improve visibility and reduce occlusion from large vehicles. The perception system uses snapshots rather than continuously processing video. The optimizer remains independent of camera count; each configured approach is represented by a standardized traffic input after perception.

6.2 Five-Second Look-Ahead

Example:

North = current GREEN

North remaining = 5 sec

Capture:

East snapshot

South snapshot

West snapshot

YOLO → counts → CEU + waiting time

→ future green durations

→ current North's upcoming red duration

6.3 YOLO Output

YOLO receives each snapshot and detects relevant vehicle classes. The raw detection information can include class, confidence, and bounding box. The system then converts detections into class-wise counts. The optimizer never consumes raw pixels; it consumes the standardized traffic state produced by the perception and traffic-processing layers.

6.4 CEU

Vehicle	Prototype Weight
Car	1.0 CEU
Bike	0.5 CEU
Bus	2.0 CEU
Truck	2.0 CEU

Example: 10 cars + 4 bikes + 2 buses + 1 truck = $10(1.0) + 4(0.5) + 2(2.0) + 1(2.0) = 18$ CEU.

The vehicle weights are configurable prototype parameters. The optimizer does not separately optimize cars, bikes, buses, and trucks; they are normalized into a single traffic-demand representation.

7. Traffic State and Waiting Time

For each approach, the traffic state contains at minimum CEU/traffic demand and directional waiting time. Waiting time is defined as the elapsed time since that direction was last served by green. When a direction receives green, its waiting timer resets; other red directions continue accumulating waiting time.

Example traffic state:

North: CEU = 12, Waiting = 0 sec
East: CEU = 25, Waiting = 15 sec
South: CEU = 35, Waiting = 8 sec
West: CEU = 8, Waiting = 31 sec

8. Optimization Engine

The optimization engine is a custom mathematical decision layer. It is not a neural network. Its purpose is to convert the standardized traffic state into a valid future signal timing plan.

8.1 Priority Calculation

Each direction receives a priority derived from traffic demand and waiting time. Conceptually: $\text{Priority} = f(\text{CEU}, \text{Waiting Time})$. The exact formula and weighting coefficients are a design parameter to be finalized and experimentally tuned.

8.2 Green-Time Allocation

The optimizer allocates future green durations subject to the configured minimum and maximum bounds. The 30-second reference duration acts as a normal baseline rather than a mandatory final duration.

8.3 Saved-Time Redistribution

If a direction requires less than its reference allocation, the resulting saved time may be redistributed to directions with greater priority, provided their green duration does not exceed the maximum. If redistribution is not useful, the total cycle may become shorter.

8.4 Red-Time Calculation

Red duration is not independently guessed. Once future green durations are determined, the upcoming red interval for a direction is derived from the future green phases and the applicable yellow/all-red transition intervals before that direction receives green again.

8.5 Frozen Timing Plan

Once a future phase duration has been validated and accepted, that duration is frozen for execution. Traffic changes during the active phase do not trigger a recalculation or interruption. Fresh data is collected again at the next five-second look-ahead trigger.

9. Signal Controller

The signal controller is a deterministic state machine responsible for executing the validated timing plan. It preserves the configured round-robin sequence and manages signal-state transitions.

```
GREEN
↓ timer expires
YELLOW
↓ timer expires
ALL-RED
↓ timer expires
NEXT GREEN
```

The controller owns execution and safety-critical state. The YOLO/optimizer pipeline proposes timing values but cannot directly bypass controller validation.

10. Safety, Validation, and Failure Handling

10.1 Validation Rules

- Every green duration must be at least the configured minimum.
- Every green duration must not exceed the configured maximum.
- The timing plan must contain valid configured approaches.
- Round-robin phase order must remain unchanged.
- No conflicting green state may be produced.
- Transition sequencing must remain GREEN → YELLOW → ALL-RED → next GREEN.
- Numeric values must be finite and valid.
- The plan must be generated within the two-second deadline.

10.2 Hard Timeout

The optimization pipeline has a strict two-second deadline beginning at the five-second trigger. If YOLO, image handling, optimization, or another required processing stage prevents a valid timing plan from being returned within the deadline, the calculation is aborted and the controller uses the predefined safe fallback timing.

10.3 Failure Cases

Failure	Required behavior
Camera/snapshot unavailable	Do not wait indefinitely; abort optimization and use fallback timing.
Invalid/corrupt image	Reject image; if no valid optimization can complete within the deadline, use fallback.
YOLO error	Abort current optimization attempt and use fallback.
YOLO timeout	Abort at 2.0 seconds and use fallback.
Optimizer exception	Reject the calculation and use fallback.
Optimizer returns invalid duration	Safety validation rejects plan; use fallback.
Communication/backend failure	Controller remains responsible for safe signal execution; use predefined fallback behavior as r

11. Backend and Dashboard

11.1 Backend

FastAPI provides the application API boundary. The backend exposes system state and operational information without becoming the authority for safety-critical signal transitions.

11.2 Dashboard

- Current direction and current signal state.
- Remaining green/red countdown.
- Estimated next green duration.
- Estimated next red duration.
- CEU per approach.
- Waiting time per approach.
- Camera status.
- YOLO status.
- Optimizer status.
- Controller/safety status.
- Fallback status when applicable.

The dashboard is primarily a monitoring interface. It should not provide arbitrary manual signal control that can bypass the controller's safety rules.

12. Technology Stack

Layer	Technology	Role
Programming	Python	Primary implementation language
Deep Learning / CV	Ultralytics YOLO	Vehicle detection from snapshots
ML Framework	PyTorch only when directly working with the training data	Deep learning for YOLO
Traffic Processing	Python	Counting, CEU, waiting time, traffic state
Optimization	Custom Python algorithm	Priority, green allocation, redistribution, red calculation
Signal Control	Python state machine	Phase execution and transitions
Safety	Deterministic Python validation	Constraints, timeout, fallback
Backend	FastAPI	API and application interface
Dashboard	React + TypeScript	Monitoring UI
Configuration	YAML / JSON	Intersection and timing parameters
Testing	pytest	Automated software testing
Version Control	Git / GitHub	Source control and collaboration

13. Implementation Architecture

```
traffic-signal-optimizer/  
├── perception/  
│   ├── yolo_detector.py  
│   ├── snapshot_manager.py  
│   └── vehicle_counter.py
```



```

|
├── traffic/
│   ├── ceu_engine.py
│   ├── waiting_time.py
│   └── traffic_state.py
├── optimizer/
│   ├── priority.py
│   ├── timing_optimizer.py
│   ├── green_allocator.py
│   ├── redistribution.py
│   └── red_calculator.py
├── controller/
│   ├── signal_controller.py
│   ├── phase_manager.py
│   ├── state_machine.py
│   └── safety.py
├── hardware/
│   └── signal_interface.py
├── backend/
│   ├── main.py
│   ├── routes/
│   └── services/
├── dashboard/
├── config/
│   ├── intersection.yaml
│   ├── timing.yaml
│   └── vehicle_weights.yaml
├── logs/
└── tests/

```

13.1 Component Responsibilities

Component	Responsibility
Snapshot Manager	Trigger and collect images from configured approaches.
YOLO Detector	Detect vehicles and return detection metadata.
Vehicle Counter	Convert detections into class-wise counts.
CEU Engine	Convert class counts into standardized CEU demand.
Waiting Time	Maintain per-direction waiting timers.
Traffic State	Build the standardized state passed to optimization.
Priority	Calculate relative priority from demand and waiting time.
Green Allocator	Determine green durations within constraints.
Redistribution	Distribute useful saved time to higher-priority directions.
Red Calculator	Derive upcoming red intervals from future phases.
Safety	Validate plan, enforce deadline, and select fallback when required.
Signal Controller	Execute validated phase states and durations.
Hardware Interface	Allow software controller to drive a future physical controller.

Component	Responsibility
Backend	Expose operational state and APIs.
Dashboard	Display system status and timing information.

14. End-to-End Runtime Sequence

- 1 A configured intersection starts in its predefined round-robin sequence.
- 2 The signal controller activates the current direction's green phase using its approved duration.
- 3 The current phase duration remains frozen while it is active.
- 4 The controller monitors the remaining green time.
- 5 When approximately five seconds remain, the optimization trigger fires.
- 6 Snapshots are captured for all upcoming/other directions.
- 7 Snapshots are passed to YOLO.
- 8 YOLO detects and classifies vehicles.
- 9 Class-wise vehicle counts are generated.
- 10 Counts are converted into CEU.
- 11 Direction-level waiting times are read from the traffic-state engine.
- 12 The optimizer constructs priorities from CEU and waiting time.
- 13 Future green durations are allocated within the minimum/maximum constraints.
- 14 Useful saved time is redistributed where appropriate.
- 15 Upcoming red durations are derived from the resulting future schedule and transition intervals.
- 16 The safety layer validates the complete timing plan.
- 17 The system checks that the result was produced within two seconds of the trigger.
- 18 If valid, the plan is accepted and future durations are frozen.
- 19 If invalid or late, the optimization attempt is aborted and the predefined fallback timing is used.
- 20 The current phase finishes, followed by yellow and all-red transitions.
- 21 The next direction receives its approved green duration.
- 22 At that phase's final five seconds, fresh snapshots are captured and the process repeats.

15. Operational Cases

Case	System behavior
Equal traffic	Maintain a balanced allocation subject to the timing rules.
One direction has very high CEU	Give that direction higher priority but never exceed the maximum green.
One direction has low CEU but very high waiting time	Waiting time priority can protect it from being repeatedly underserved.
Low traffic on all approaches	Use shorter allocations where appropriate rather than unnecessarily preserving the reference cycle.
High traffic on all approaches	Allocate timing according to relative priority while respecting maximum limits and round-robin order.

Case	System behavior
Saved time exists	Redistribute to higher-priority directions when useful; otherwise shorten the overall cycle.
Traffic changes during green	Do not recalculate or interrupt the active phase; use the next five-second trigger.
YOLO produces valid result quickly	Continue through CEU, optimization, validation, and execution.
YOLO/optimizer exceeds two seconds	Abort optimization and use fallback timing.
Invalid optimization result	Reject through safety validation and use fallback.
Physical hardware unavailable	Run through a software signal interface; hardware remains optional.

16. Testing Architecture

Testing is layered so that perception accuracy, deterministic traffic calculations, optimization correctness, safety behavior, controller state transitions, and the complete pipeline can be validated independently.

Level	Focus
Unit	CEU, waiting time, priority, green allocation, redistribution, red calculation, validation.
YOLO evaluation	Precision, recall, mAP, class-wise performance, inference latency, and downstream CEU impact.
Component	Perception, traffic state, optimizer, controller, safety modules in isolation.
Integration	YOLO → counts → CEU → waiting time → optimizer → safety → controller.
End-to-End	Snapshot through executed timing and dashboard state.
Performance	Decision latency against the two-second deadline.
Fault injection	YOLO failure, image failure, optimizer failure, invalid output, communication failure.
Regression	Run the full automated suite after optimizer/controller changes.
Hardware	Optional ESP32/LED prototype timing, transitions, communication failure, restart, long-running behavior.

17. Non-Functional Requirements

Category	Requirement
Latency	Optimization pipeline must produce a valid plan within 2.0 seconds of the five-second trigger or fall back.
Safety	Invalid, late, or missing optimization results must never bypass deterministic validation.
Reliability	Signal execution must not depend on continuous availability of YOLO or the dashboard.
Explainability	Timing decisions should be reproducible from traffic state, configuration, and optimizer outputs.
Configurability	Intersection topology, timing boundaries, transitions, and vehicle weights should be configurable.
Maintainability	Perception, traffic state, optimization, safety, controller, backend, and UI should remain separated.
Observability	Optimization inputs, outputs, timing, validation, and fallback events should be logged.
Extensibility	Future enhancements should be addable without replacing the signal-controller foundation.

18. Evaluation and Success Metrics

The project should be evaluated internally by comparing the optimized signal behavior against a conventional fixed-timing baseline under controlled traffic scenarios. This benchmark is for

development, validation, tuning, and demonstration rather than a client-facing dashboard feature.

- Average waiting time.
- Total waiting time.
- Traffic throughput.
- Average cycle duration.
- Time saved relative to the fixed baseline.
- Optimizer decision latency.
- YOLO detection performance.
- Percentage of optimization cycles completed successfully within the two-second deadline.
- Fallback frequency and failure causes.

19. Recommended Implementation Order

- 1 Create the intersection and approach configuration model.
- 2 Implement the traffic signal state machine and fixed round-robin controller.
- 3 Implement waiting-time tracking.
- 4 Implement snapshot capture interfaces.
- 5 Integrate YOLO inference and vehicle counting.
- 6 Implement CEU conversion.
- 7 Create the standardized traffic-state object/schema.
- 8 Design and implement the priority calculation.
- 9 Implement green-time allocation with minimum/reference/maximum constraints.
- 10 Implement saved-time redistribution.
- 11 Implement red-duration calculation.
- 12 Implement timing-plan validation.
- 13 Implement the strict two-second timeout and fallback mechanism.
- 14 Integrate the complete perception-to-controller pipeline.
- 15 Add FastAPI backend.
- 16 Build the monitoring dashboard.
- 17 Add structured logging and observability.
- 18 Implement automated tests and failure-injection tests.
- 19 Add the optional hardware interface and physical prototype after software validation.

20. Future Enhancement Roadmap

The project is treated as one complete planned system. Future enhancements are optional additions that can be implemented whenever time, data, hardware, or research scope allows; they are not separate mandatory versions.

Enhancement	Potential value
Queue length	Distinguish simple vehicle counts from dense queues and improve demand estimation.
Historical traffic	Use time-of-day/day-of-week patterns to anticipate demand.
Predictive models	Estimate near-future traffic rather than relying only on current snapshots.
Reinforcement learning	Explore learned policies after sufficient simulation/data infrastructure exists.
Multi-intersection coordination	Optimize neighboring signals as a network.
Sensor fusion	Combine cameras with additional traffic sensors.
Physical deployment	Connect the controller to real signal hardware after appropriate safety engineering.

21. Final System Summary

The Traffic Signal Optimizer is a layered Deep Learning and Computer Vision system in which YOLO provides the perception capability and a custom mathematical optimization engine provides the adaptive traffic decision-making. The system observes traffic through periodic snapshots, converts vehicle detections into CEU, combines CEU with direction-level waiting time, dynamically allocates future green time, derives the corresponding red duration, validates the timing plan, and executes it through a deterministic round-robin signal controller.

The central runtime rule is simple: the active phase is stable and frozen; at approximately five seconds remaining, the system looks ahead, captures the upcoming approaches, computes a new plan, and prepares it for the next phases. Safety remains outside the learned perception model and is enforced through explicit constraints, a strict two-second timeout, and predefined fallback behavior. This separation keeps the project technically defensible as a Deep Learning + Computer Vision application while maintaining a clear, explainable, and testable control architecture.

Reference architecture principle

YOLO sees. The traffic engine quantifies. The optimizer decides. The safety layer validates. The controller executes. The dashboard observes.